Search... | 2 Offers Ending | Courses | Tutorials | Practice | Jobs

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Input:

arr[] =

7 10 4 3 20 15

k =

3

Your Output:

7

Expected Output:

7

Java (21) | Start Timer

```java
import java.util.Arrays;

class Solution {
    public int kthSmallest(int[] arr, int k) {
        Arrays.sort(arr);
        return arr[k - 1];
    }
}
```

Custom Input | Compile & Run | Submit

Search... | 2 Offers Ending | Courses | Tutorials | Practice | Jobs

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Input:

k =

2

arr[] =

1 5 8 10

Your Output:

5

Expected Output:

5

Java (21) | Start Timer

```java
import java.util.Arrays;

class Solution {
    public int getMinDiff(int[] arr, int k) {
        int n = arr.length;
        Arrays.sort(arr);

        int ans = arr[n - 1] - arr[0];

        int smallest = arr[0] + k;
        int largest = arr[n - 1] - k;

        for (int i = 1; i < n; i++) {
            int min = Math.min(smallest, arr[i] - k);
            int max = Math.max(largest, arr[i - 1] + k);

            if (min < 0) continue;

            ans = Math.min(ans, max - min);
        }
        return ans;
    }
}
```

Custom Input | Compile & Run | Submit

Search... | Courses | Tutorials | Practice | Jobs

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Input:

arr[] =

1 3 5 8 9 2 6 7 6 8 9

Your Output:

3

Expected Output:

3

Java (21) | Start Timer

```java
class Solution {
    public int minJumps(int[] arr) {

        int n = arr.length;

        if (n <= 1) return 0;
        if (arr[0] == 0) return -1;

        int maxReach = arr[0];
        int steps = arr[0];
        int jumps = 1;

        for (int i = 1; i < n; i++) {

            if (i == n - 1)
                return jumps;

            maxReach = Math.max(maxReach, i + arr[i]);
            steps--;

            if (steps == 0) {
                jumps++;

                if (i >= maxReach)
                    return -1;

                steps = maxReach - i;
            }
        }

        return -1;
    }
}
```

Custom Input | Compile & Run | Submit

Problem List | Submit | 0 | 01:19:59 | Premium

Description | Editorial | Solutions | Submissions

287. Find the Duplicate Number

Medium | Topics | Companies

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.

There is only **one repeated number** in `nums`, return *this repeated number*.

You must solve the problem **without** modifying the array `nums` and using only constant extra space.

Example 1:

```
Input: nums = [1,3,4,2,2]
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
Output: 3
```

Example 3:

```
Input: nums = [3,3,3,3,3]
Output: 3
```

25.3K | 501 | 161 Online

Code

Java | Auto

```java
        // Detect cycle
        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while (slow != fast);

        // Find entry point of cycle
        slow = nums[0];
        while (slow != fast) {
```

Saved | Ln 21, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

nums =
[1,3,4,2,2]

Output

2

Search... | 2 Offers Ending | Courses | Tutorials | Practice | Jobs

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Input:

a[] =
1 3 5 7

b[] =
0 2 6 8 9

Your Output:

[0, 1, 2, 3]
[5, 6, 7, 8, 9]

Expected Output:

[0, 1, 2, 3]

Java (21) | Start Timer

```java
class Solution {
    public void mergeArrays(int a[], int b[]) {
        int n = a.length, m = b.length;
        int gap = nextGap(n + m);

        while (gap > 0) {
            int i = 0, j = gap;

            while (j < n + m) {
                // i in a[], j in a[]
                if (i < n && j < n) {
                    if (a[i] > a[j]) {
                        int temp = a[i];
                        a[i] = a[j];
                        a[j] = temp;
                    }
                }
                // i in a[], j in b[]
                else if (i < n && j >= n) {
                    if (a[i] > b[j - n]) {
                        int temp = a[i];
                        a[i] = b[j - n];
                        b[j - n] = temp;
                    }
                }
                // i in b[], j in b[]
                else {
                    if (b[i - n] > b[j - n]) {
                        int temp = b[i - n];
                        b[i - n] = b[j - n];
                        b[j - n] = temp;
                    }
                }
                i++;
                j++;
```

Custom Input | Compile & Run | Submit

Problem List | Submit | 0 | 01:19:59 | Premium

Description | Editorial | Solutions | Submissions

56. Merge Intervals

Medium | Topics | Companies

Given an array of `intervals` where `intervals[i] = [start$_i$, end$_i$]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].
```

Example 2:

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
Explanation: Intervals [1,4] and [4,5] are considered overlapping.
```

Example 3:

```
Input: intervals = [[4,7],[1,4]]
Output: [[1,7]]
Explanation: Intervals [1,4] and [4,7] are considered overlapping.
```

24.5K | 275 | 353 Online

Code

Java | Auto

```java
import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        if (intervals.length == 0) return new int[0][];

        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

        List<int[]> res = new ArrayList<>();
        int[] curr = intervals[0];
```

Saved | Ln 24, Col 2

Testcase | Test Result

Accepted Runtime: 1 ms

Case 1 | Case 2 | Case 3

Input

intervals =
[[1,3],[2,6],[8,10],[15,18]]

Output

[[1,6],[8,10],[15,18]]

Search... | Courses | Tutorials | Practice | Jobs

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Input:

```
1 5 10 20 40 80
6 7 20 80 100
3 4 15 20 30 70 80 120
```

Your Output:

20 80

Expected Output:

20 80

Java (21) | Start Timer

```java
import java.util.*;

class Solution {
    public List<Integer> commonElements(List<Integer> arr1,
                                        List<Integer> arr2,
                                        List<Integer> arr3) {
        List<Integer> res = new ArrayList<>();
        int i = 0, j = 0, k = 0;

        while (i < arr1.size() && j < arr2.size() && k < arr3.size()) {
            int a = arr1.get(i);
            int b = arr2.get(j);
            int c = arr3.get(k);

            if (a == b && b == c) {
                res.add(a);
                i++; j++; k++;
            }
            else if (a < b) i++;
            else if (b < c) j++;
            else k++;
        }

        if (res.isEmpty()) {
            res.add(-1);
        }

        return res;
    }
}
```

Search... | Courses | Tutorials | Practice | Jobs

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Input:

a[] =

```
11 7 1 13 21 3 7 3
```

b[] =

```
11 3 7 1 7
```

Your Output:

true

Expected Output:

true

Java (21) | Start Timer

```java
import java.util.*;

class Solution {
    public boolean isSubset(int a[], int b[]) {
        HashSet<Integer> set = new HashSet<>();

        for (int x : a) {
            set.add(x);
        }

        for (int x : b) {
            if (!set.contains(x)) {
                return false;
            }
        }

        return true;
    }
}
```

Custom Input | Compile & Run | Submit

Search... | Courses | Tutorials | Practice | Jobs

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Input:

n

```
5
```

Your Output:

1 2 0

Expected Output:

1 2 0

Java (21) | Start Timer

```java
import java.util.*;

class Solution {
    public static ArrayList<Integer> factorial(int n) {
        ArrayList<Integer> res = new ArrayList<>();
        res.add(1);   // 1! = 1

        for (int i = 2; i <= n; i++) {
            multiply(res, i);
        }

        Collections.reverse(res);
        return res;
    }

    private static void multiply(ArrayList<Integer> res, int num) {
        int carry = 0;

        for (int i = 0; i < res.size(); i++) {
            int prod = res.get(i) * num + carry;
            res.set(i, prod % 10);
            carry = prod / 10;
        }

        while (carry > 0) {
            res.add(carry % 10);
            carry /= 10;
        }
    }
}
```

Search... | 2 Offers Ending | Courses | Tutorials | Practice | Jobs

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Input:

arr[] =

1 4 45 6 10 8

target =

13

Your Output:

true

Expected Output:

true

Java (21) | Start Timer

```java
import java.util.*;

class Solution {
    public boolean hasTripletSum(int arr[], int target) {
        Arrays.sort(arr);
        int n = arr.length;

        for (int i = 0; i < n - 2; i++) {
            int left = i + 1, right = n - 1;

            while (left < right) {
                int sum = arr[i] + arr[left] + arr[right];

                if (sum == target)
                    return true;
                else if (sum < target)
                    left++;
                else
                    right--;
            }
        }
        return false;
    }
}
```

Custom Input | Compile & Run | Submit

Search... | 2 Offers Ending | Courses | Tutorials | Practice | Jobs

Problem | Editorial | Submissions | Comments

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Input:

arr[] =

3 0 1 0 4 0 2

Your Output:

10

Expected Output:

10

Java (21) | Start Timer

```java
class Solution {
    public int maxWater(int arr[]) {
        int left = 0, right = arr.length - 1;
        int leftMax = 0, rightMax = 0;
        int water = 0;

        while (left < right) {
            if (arr[left] <= arr[right]) {
                if (arr[left] >= leftMax)
                    leftMax = arr[left];
                else
                    water += leftMax - arr[left];
                left++;
            } else {
                if (arr[right] >= rightMax)
                    rightMax = arr[right];
                else
                    water += rightMax - arr[right];
                right--;
            }
        }
        return water;
    }
}
```